



TSUNAMI: a GPU implementation of the WFA algorithm

Giulia Gerometta, Alberto Zeni, Marco D. Santambrogio

Dipartimento di Elettronica, Informatica e Bioingegneria, Politecnico di Milano, Milan, Italy

{giulia.gerometta}@mail.polimi.it

{alberto.zeni, marco.santambrogio}@polimi.it

Abstract—Pairwise sequence alignment represents a fundamental step in the genome assembly pipeline, being the most time-consuming step and the bottleneck factor in multiple bioinformatics applications. Exact pairwise alignment methods like Smith–Waterman and Needleman–Wunsch, often cannot satisfy the performance required by these tools because of their quadratic time complexity. Furthermore, given the increasing computational cost of analyzing *third-generation* sequences, the community is moving towards different alignment methods and hardware-accelerated solutions to overcome the limitations of these algorithms. In this scenario, we present TSUNAMI, a highly-optimized implementation of the WaveFront Alignment (WFA) algorithm on GPU. TSUNAMI exploits GPU high-parallel computing to accelerate the WFA algorithm, a novel alignment methodology exploiting homologous regions between the target sequences. By doing so, we are able to reduce both time and space complexity in our GPU implementation. Our results show that TSUNAMI achieves improvements up to $4512.28\times$ in terms of speedup when compared to the multi-threaded state-of-the-art software implementation run on Intel Xeon Silver 4208 using 16 threads in total. We also compared our design with all the recently released hardware-accelerated solutions present in the State Of the Art, observing speedups up to $14.81\times$ with respect to the best performing hardware-accelerated implementation in the literature, reaching up to 42604.98 Giga Cell Updates Per Second in our best configuration. TSUNAMI also supports aligning very erroneous long sequences, rendering our implementation much more useful in real-world scenarios. Finally, to prove the efficiency of our design, we evaluate TSUNAMI exploiting the Berkeley Roofline model and demonstrate that our implementation is near-optimal on the NVIDIA Tesla H100.

Index Terms—HPC, GPU, Genome Alignment, WFA

I. INTRODUCTION

Pairwise sequence alignment is one of the most commonly used methodologies to identify and analyze relationships between -omics sequences. Points of similarity or difference between sequences might indicate deviations in the functions of genes and also highlight mutations [1]. Therefore, pairwise sequence alignment is one of the core components of numerous bioinformatics pipelines, such as genome assembly [2], [3], read mapping [4]–[6], and variant calling [7], [8]. Most of the aforementioned applications are based upon optimal alignment algorithms, namely the Smith–Waterman (SW) [9] and Needleman–Wunsch (NW) [1] algorithms since they always report the optimal matching between the two target sequences. Moreover, the community often prefers extensions to these algorithms, such as the Smith–Waterman–Gotoh (SWG) algorithm [10]. Indeed, this additional optimal method

built upon SW adds the support for affine gap-insertion, producing results often preferred by bioinformaticians given the additional information provided by this kind of analysis. However, optimal alignment algorithms require quadratic time and space complexity. Hence, these algorithms result to be particularly expensive, especially when aligning *third-generation* sequences due to the high computational cost of analyzing very long sequences (of over 1K bases). For this reason, many research efforts focus on improving the performance of sequence alignment algorithms and their respective variations. To improve upon this methodology, *Marco-Sola et al.* [11] recently proposed the WFA algorithm, an extension of the Myers [12], [13] algorithm, which avoids the full quadratic execution time of SWG while still producing the same results. WFA exploits homologous regions of the target sequences to reduce the number of operations required to align the sequences, making it a very promising candidate for replacing the SWG algorithm in genomics pipelines, especially when the similarity of the two analyzed sequences is very high. Indeed, the asymptotic bounds of time and space complexity in WFA directly correlates with the similarity of the two sequences, being $O(n \times s)$ and $O(s^2)$ respectively, where n the sequence length and s the distance score; as opposed to SWG, SW and NW, for which the computation of the optimal solutions requires $O(m \times n)$ time and $O(m \times n)$ memory (with m and n the lengths of the two aligned sequences).

Researchers often rely on heterogeneous architectures to speed up and parallelize the alignment process to further optimize these algorithms' performance. Indeed, hardware accelerators play an enormous role in the genomics context and are often exploited to overcome the limitations of general-purpose architectures, such as CPUs and the previously mentioned optimal algorithms. Modern genomics applications often demand more than what current processors can deliver, as they cannot efficiently analyze the constantly increasing amount of data these algorithms require to process. In the State Of the Art, multiple solutions, notably [14]–[21], exploit heterogeneous architectures, like Graphics Processing Units (GPUs) and Field Programmable Gate Arrays (FPGAs), to speed up the alignment process of popular genome alignment algorithms. Despite the multiple FPGA- and GPU-accelerated solutions of SW, SWG, and NW in the State Of the Art, their quadratic complexity and expensive computational cost implies that they are still rarely integrated into high-impact genomics pipelines.

By contrast, only a few implementations in the literature tackle the acceleration of the WFA, often lacking flexibility and performance. Indeed, these implementations [22]–[24] support only alignments with difference rates between the sequences that are lower than 10%, while often supporting only the alignment of short sequences [22], [23]. These compromises are due to the increased complexity of implementing WFA efficiently on heterogeneous architectures, compared with NW and SW methods, because of the dynamic and complex nature of its computation. Indeed, given the very dynamic nature of WFA, which often requires a wide range of allocated resources depending on the characteristics of the aligned sequences, previously presented implementations often introduce significant limitations in the error rates and sequence lengths supported, making their integration within genomic pipelines practically unfeasible. In this context, we propose TSUNAMI, a flexible, highly-optimized GPU implementation of the WFA algorithm. Our design remarkably outperforms the software implementation (WFA2-lib [25]) when run in parallel on 16 threads, achieving up to $4512.28\times$ speedup. Furthermore, our design also outperforms all the other hardware accelerators of the WFA algorithm in the literature both in terms of flexibility and throughput. In particular, our solution attains speedups of up to $14.81\times$ when compared with the best-performing FPGA design in the literature [23] and up to $17.04\times$ with respect to the State Of the Art GPU acceleration of this algorithm [24]. Our implementation is also the first hardware-accelerated design of WFA that is able to dynamically and efficiently adjust the number of allocated resources at runtime, enabling the alignment of long and very erroneous sequences on heterogeneous hardware, with error rates over 10% and read lengths of over 10K bases.

The main contributions introduced by this work are:

- the best performing GPU implementation of the WFA algorithm, named TSUNAMI, which achieves significant speedups over leading versions of the same algorithm, run on state-of-the-art processors and hardware accelerators, showing improvements in runtimes ranging from $14.81\times$ to $4512.28\times$.
- the first hardware-accelerated implementation of WFA, which supports the alignment of sequences with error rates higher than 10% and read lengths of over 10K bases.
- an adaptation of the Berkeley Roofline Model used to evaluate TSUNAMI and underlying hardware, which demonstrates that the performance of our design is near optimal on NVIDIA Tesla H100.

The remainder of the paper is organized as follows. Section III provides an overview of the related work, while Section II briefly introduces the algorithm we optimized to run on GPU. Section IV details the design of TSUNAMI, highlighting the various optimizations and features we implemented. Section V illustrates our experimental results, Section VI describes the evaluation we made on TSUNAMI using the Berkeley Roofline model and finally, Section VII summarizes

the contributions of this work and future developments.

II. WAVEFRONT ALIGNMENT ALGORITHM

This section aims to describe the WFA algorithm proposed by *Marco-Sola et al.* [11]. WFA is a novel pairwise sequence alignment algorithm that extends the Myers' one [12] to the gap-affine model. The optimal SW and NW algorithms often employ gap-linear scoring. However, the most widely used scoring function for sequence analysis relies on the gap-affine scoring method, which introduces gap opening and gap extension penalties, accounting for different and incremental costs for the deletions and insertions within the alignment. Gotoh [10] applied the gap-affine model to the SW algorithm. The recurrence equation of the SWG is the following:

$$\begin{cases} I_{v,h} = \min\{M_{v,h} + o + e, I_{v,h-1} + e\} \\ D_{v,h} = \min\{M_{v-1,h} + o + e, D_{v-1,h} + e\} \\ M_{v,h} = \min\{I_{v,h}, D_{v,h}, M_{v-1,h-1} + s(q_{v-1}, th - 1)\} \end{cases} \quad (1)$$

SWG Dynamic Programming (DP)-approach relies on three matrices storing the optimal subsolutions: M , I , and D . An entry $M(i, j)$ caches the best score for the alignment of $m(0, i)$ with $n(0, j)$ subsequences. While cells $D(i, j)$ and $I(i, j)$ contain the best score under the condition that the alignment ends in a gap within sequence n and m , respectively. In the sequence alignment context, one of the most employed algorithms in the State Of the Art is the Myers' [12], [13]. The Myers' algorithm computes the longest common subsequence of two strings exploiting extensions of matching characters along the diagonals of a so-called *edit graph*. Myers algorithm constructs an edit graph by placing the *source* sequence on the x -axis and the *destination* sequence along the y -axis to form a grid. The alignment of the sequences starts from the upper-left edge of the constructed grid, and each move towards a vertex corresponds to an *edit* operation on the source sequence. A diagonal shift represents the equivalence between source and destination sequence elements. Meanwhile, horizontal moves correspond to deletions from the source sequence, and vertical moves denote insertions in the destination string. The final product of this algorithm is a so-called *edit score*, which denotes the similarity of the two sequences. However, this methodology fails to detail crucial properties when employed for the alignment between biological sequences, as it lacks information on the cost of extending and opening a gap provided by gap-affine aligners [26]. Hence, *Marco-Sola et al.* [11] exploit the homologous regions between the two DNA reads, following Myers' method, and employs the gap-affine model, attaining the same results of SWG with the performance improvements of Myers' algorithm. In contrast to the SWG algorithm, the WFA procedure progresses in diagonal waves instead of populating the alignment matrix cell by cell. To do so, the WFA algorithm redefines the formulation of SWG as follows. For each score s and diagonal k of the DP-matrix, the WFA algorithm defines a

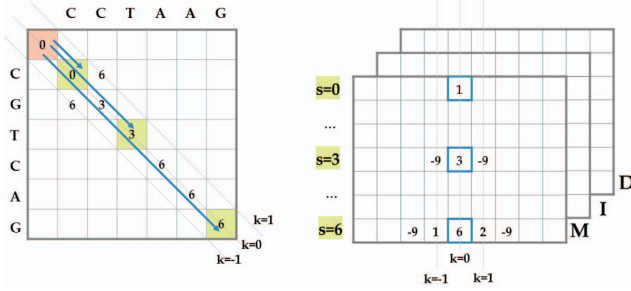


Fig. 1. Comparison between the alignment of two sequences exploiting a DP-matrix (left) and wavefronts (right). Matching information is marked with the same color.

Furthest Reaching Point (FRP). Every FRP denotes the DP-cell, which is the most far off the beginning of the diagonal following only matching characters. FRPs are pointed by the blue arrows in Figure 1. Hence, for each of the three matrices (M, I, and D) of the SWG algorithm (Equation (1)), the WFA algorithm defines the offsets ($M_{s,k}$, $I_{s,k}$, and $D_{s,k}$), for each diagonal k and score s , as the lengths of the diagonals that starting from the upper-left DP-cell end in the corresponding FRP. The algorithm then defines *wavefronts*, for each score s , as the collection of all the offsets of score s . The redefined WFA recurrence equations of the DP matrices in terms of *wavefronts* are described in the following equation:

$$\begin{cases} I_{s,k} = 1 + \max\{M_{s-o-e,k-1}I_{s-e,k-1}\} \\ D_{s,k} = \max\{M_{s-o-e,k+1}D_{s-e,k+1}\} \\ M_{s,k} = \max\{M_{s-x,k} + 1I_{s,k}D_{s,k}\} \end{cases} \quad (2)$$

To compute the missing information regarding matches, all the previously computed points are then extended following the matching characters along the diagonal. Consequently, the optimal alignment coincides with the series of *wavefronts* ending with the s wavefront, being s the smallest score, which stores a FRP reaching the end of the reads. Figure 1 shows the same alignment executed using SW DP-matrix on the left, and using the WFA algorithm on the right. Offsets are represented as the lengths of the blue arrows on the left and bordered in blue on the right. We highlighted scores in yellow in both representations. The WFA procedure progressively computes *wavefronts* of increasing scores. The WFA algorithm comprises four main phases: *Initialization*, *Extend*, *Next Wavefront* and *Backtrace*. The algorithm starts with the *Initialization* phase, where the edit score and *wavefronts* are set with a *null* score and offset, respectively. In the *Extend* phase, WFA computes each $M_{s,k}$ offset following the matching characters along the diagonals. Then, the algorithm checks if any of the calculated FRPs reached the end of the sequences. If so, the algorithm ends, proceeding to the *Backtrace* step. Otherwise, the *Next Wavefront* phase follows. The *Next Wavefront* step starts the computation of a new wavefront starting by increasing the score s . For each diagonal

Algorithm 1 WFA algorithm

Input: query q , text t , affine penalties p

Output: gap affine optimal alignment

```

 $M_{i,j} \leftarrow -10$  /*null offset*/
 $M_{0,0} \leftarrow 0$ 
 $k \leftarrow \text{len}(q) - \text{len}(t)$ 
 $\text{offset} \leftarrow \text{len}(q)$ 
 $s \leftarrow 0$ 
while true do
     $\text{EXTEND}(q, t, M_s)$ 
    if  $M_{s,k} \geq \text{offset}$  then
        break
     $s \leftarrow s + 1$ 
     $\text{NEXT\_WAVEFRONT}(M, I, D, q, t, s)$ 
 $\text{BACKTRACE}(M, I, D, q, t, s)$ 

```

k , the algorithm computes the cells within the s *wavefronts* following the WFA recurrence Equation (2). The *Extend* and *Next Wavefront* steps are repeated until a computed FRP reaches the end of the sequences, marking that an optimal alignment has been found. Once this happens, the algorithm proceeds to the *Backtrace* phase. WFA performs the backtrace across the *wavefronts*' offsets. Thus, the *Backtrace* step starts from the last computed wavefront, the one that produced the s score, and progressively calculates which FRP of the previous wavefront (i.e. $s-1$ wavefront) has generated the under-analysis offset. Algorithm 1 provides a high-level overview of the WFA algorithm. The *wavefronts* approach allows for the reduction of both spatial and time complexity. More precisely, the WFA algorithm has spatial and time complexity related to the distance score s of the alignment and sequence length n : $O(s^2)$ and $O(n \times s)$. Therefore, the WFA performs better when the two sequences undergoing the alignment are similar. Hence, the time and space complexity of the algorithm is strictly related to the *error rate* of the sequences. We define the *error rate* as the percentage of different bases between the two DNA sequences aligned by the algorithm. With *error rates* lower than 30% (found empirically), n is significantly lower than the distance score s . Otherwise, when the sequences have an *error rate* higher than this threshold, the distance score exceeds the sequence length, and as a consequence, using the SWG algorithm becomes more convenient.

III. RELATED WORK

This section provides a collection of the most relevant software and hardware solutions that leverage the WFA algorithm. Multiple tools in the State Of the Art already leverage WFA as their primary aligner algorithm. AncestralClust [27] is a phylogenetic clustering method for batching divergent sequences. The first step for building the phylogenetic tree is pairwise sequence alignment; AncestralClust exploits WFA for this task. Accel-Align [28] is a Seed-Embed-Extend-based short-read mapper and aligner that uses the WFA algorithm to perform the alignment to lower the execution time.

Song *et al.* [29] proposed AnchorWave (Anchored Wavefront alignment), which implements a whole-genome alignment algorithm that leverages genome collinearity. In this context, AnchorWave uses the WFA algorithm to align anchors (coding sequences within the genome). However, if the sequence length grows over a certain threshold, both AnchorWave [29] and Accel-Align [28] tools rely on Minimap2's *ksw* library [6] to limit memory usage. Indeed, WFA memory requirements are the limiting factor when aligning particularly long or noisy reads. For this reason, Eizenga *et al.* [30] proposed a solution that lowers the memory asymptotic bound from $O(s^2)$ to $O(s^{2/3})$. Their methodology, inspired by WhatsHap [31], retains only a subset of *wavefronts* in memory. However, the procedure needs all the computed *wavefronts* to retrieve the optimal alignment, to recompute blocks of missing *wavefronts* to perform the traceback of the alignment. Furthermore, Marco-Sola *et al.* [32] recently proposed the Bidirectional WFA (BiWFA). BiWFA is a novel implementation of the WFA algorithm computing the alignment of two reads in the forward and reverse direction until they meet. BiWFA computes the alignment in $O(s)$ memory at the expense of a higher alignment time. Concerning hardware implementations of WFA, Haghi *et al.* [22] proposed an FPGA design of the algorithm and tested their design using an IBM POWER9 CPU and two Xilinx Virtex Ultrascale Plus XCVU37)-2E. They achieved up to $8.8\times$ speedup with one FPGA and up to $13.51\times$ with two FPGAs; notably, they also execute the compact backtrack for the alignments on FPGA. However, their implementation shows multiple limitations, namely, their accelerator can only support the alignment of sequences with a fixed read length (*max 300 bps*), and they also employ 2-bit encoding of the sequences, thus limiting the aligner to only four DNA characters (A, C, G, T). Branchini *et al.* [23] recently proposed an FPGA design that overcomes these limitations and also shows significant performance improvements, reaching up to 2876 Giga Cell Updates Per Second (GCUPS) with a single Xilinx Alveo U250 FPGA board. Safaa Diab *et al.* recently proposed a framework for the Processing-in-memory (PIM) of WFA on UPMEM technology [33]. This implementation, while providing underwhelming performance results compared to other hardware-accelerated solutions, achieves significant results in terms of power efficiency. On the GPU side, three implementations of the WFA algorithm have been presented in the literature,[34] by David Castells-Rufas and eWFA-GPU [35], and WFA-GPU [24] both developed by Aguado-Puig *et al.*. The first solution adapts the WFA algorithm to compute the Levenshtein distance for very long strings. It achieves significant performance results at the expense of significantly simplifying the algorithm and removing its backtrack step. eWFA-GPU was the first implementation of WFA on GPUs and computes only the optimal edit-distance WFA between the sequences, thus limiting the design applicability in real-world scenarios. WFA-GPU, on the other hand, extends upon eWFA-GPU by fully enabling the computation of the WFA *Backtrace* step. Aguado-Puig *et al.* employ an Intel Xeon-W2155 processor and an NVIDIA GeForce RTX 3080

to speed up the execution of the WFA CPU library. This solution also proposes a *piggy-back* version for computing the alignment backtrack, which has then been integrated into the CPU WFA2-lib [25] library. Implementing the *piggy-back* methodology enables storing only a limited amount of information, thus significantly reducing the amount of *wavefronts* required for computing the alignment backtrack. The *piggy-back* backtrack procedure stores the information about the derivation of each cell at each iteration of the WFA algorithm. By doing so, WFA-GPU is able to retain only four *wavefronts* in memory, allowing the execution of more alignments in parallel. However, this implementation only supports alignments between very similar sequences, offloading to the CPU tasks that require the alignment of sequences with more than a 10% error rate, leading to sub-optimal performance results. TSUNAMI is able to overcome this limitation, as it is capable of aligning sequences with error rates bigger than 10% without offloading any task to the CPU, and it also achieves significantly better performance in every configuration supported by WFA-GPU.

IV. IMPLEMENTATION

In this Section, we describe the design and the optimizations we applied to TSUNAMI, our GPU optimized design of the WFA algorithm. The key aspect of our implementation is fully exploiting the various parallel patterns of WFA while maximizing the available resources on our target GPUs. Firstly, we focused on parallelizing the execution of each WFA instance, targeting the various steps of its execution. In particular, we were able to exploit intra-task parallelism scheduling GPU threads to parallelize the *Initialization* and *Extend* steps of the alignment as well as the computation of the *Next Wavefront*. To achieve inter-task parallelism, we implement the parallel execution of multiple pairwise alignments by scheduling one GPU block per alignment. Finally, to avoid expensive memory transfers, we also compute the *Backtrace* step of the alignment on the GPU. The next Sections are organized as follows: Section IV-A and Section IV-B detail the implementation of TSUNAMI, focusing on the intra- and inter-parallelization of the WFA algorithm respectively, Section IV-C shows how we integrated the backtrack step inside the GPU kernel.

A. Intra-task parallelism

We first focused on exploiting intra-task parallelization, meaning the parallelization of each single alignment instance. The WFA algorithm starts with initializing the wavefronts with null offsets. To speed up this process, we map multiple GPU threads to set a *null* offset to all the elements within the wavefronts. Following the *Initialization* phase, the *Extend* step is in charge of computing the extension of the previously attained FRPs along the matching characters (i.e., computing the *offset*) and stretching the FRPs with the *offsets* produced for each diagonal k . We schedule multiple GPU threads to compare chunks of characters of the two target reads in parallel, and if needed, we iterate this procedure to process all the analyzed characters at the given instance. We cache the

Algorithm 2 EXTEND parallel algorithm

Input: query q , text t

```
 $tid \leftarrow threadIdx$   
 $equal\_chars \leftarrow 0$   
 $res[tid] \leftarrow 0$   
synchronize_threads()  
for ( $i \leftarrow 1$  to  $blockDim$ ) do  
  if ( $tid \bmod 2 * i == 0$ ) then  
     $res[tid] \leftarrow res[tid] + res[tid + i]$   
   $i \leftarrow i * 2$   
synchronize_threads()  
 $equal\_chars \leftarrow res[0]$ 
```

Algorithm 3 NEXT WAVEFRONT parallel algorithm

Input: matrix M , matrix I , matrix D , score s

```
 $hi \leftarrow \max\{M_{s-x}^{hi}, M_{s-o-e}^{hi}, I_{s-e}^{hi}, D_{s-e}^{hi}\} + 1$   
 $lo \leftarrow \min\{M_{s-x}^{lo}, M_{s-o-e}^{lo}, I_{s-e}^{lo}, D_{s-e}^{lo}\} - 1$   
 $tid \leftarrow threadIdx$   
for ( $k \leftarrow lo$  to  $hi$ ) do  
   $idx \leftarrow tid + k$   
  if  $idx \geq (hi - lo)$  then  
     $I_{s,k+idx} \leftarrow \max\{M_{s-o-e,k-1+idx}, I_{s-e,k-1+idx}\} +$   
1  
     $D_{s,k+idx} \leftarrow \max\{M_{s-o-e,k+1+idx}, D_{s-e,k+1+idx}\}$   
     $M_{s,k+idx} \leftarrow \max\{M_{s-x,k+idx} +$   
1,  $I_{s,k+idx}, D_{s,k+idx}\}$   
   $k \leftarrow k + blockDim$ 
```

comparison results in shared memory to reduce the overall latency of the *Extend* step. Then, we count the occurrences of the matching characters employing a parallel reduction mechanism, exploiting in-warp GPU operations to better speed up this task. Algorithm 2 provides a high-level overview of our implementation of the *Extend* function. The naive execution approach would require a time complexity of $O(n)$ (i.e., $n = \text{sequence length}$) to compare the sequence characters; our approach reduces it to $O(\frac{n}{P} + \log(P))$ (i.e., $P = \text{number of threads}$). The *Next Wavefront* function increments the distance score and computes the cells of the s -wavefront (i.e., the current wavefront). This procedure iterates on the diagonal k , which computes the elements within the s wavefront. Analyzing the dependencies between the three matrices storing wavefronts (i.e., M , I , D), we noticed that every iteration of the loop does not depend on a previous or on a future one. Therefore, we proceeded to unroll the loop, leveraging GPU threads to execute all the iterations of the loop in parallel. Thus, for each score s , each GPU thread within the block computes one diagonal k independently, as shown in Figure 2. Our parallel implementation is resumed in Algorithm 3.

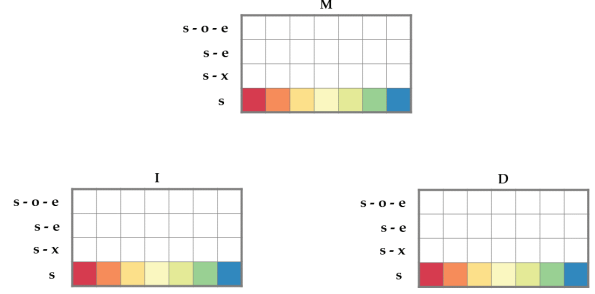


Fig. 2. Parallelization of the *Next Wavefront* step. Colored cells represent GPU threads. Cells with the same color are computed by the same thread.

B. Inter-task parallelism

To exploit inter-task parallelism, we mapped one alignment per GPU block; by doing this, we were able to compute multiple pairs of sequences in parallel while also better exploiting the GPU resources. Hence, TSUNAMI dynamically schedules GPU blocks according to the target GPU's resources. In our experiments, we used three different GPUs, the NVIDIA RTX 3080 (GeForce series), NVIDIA A100, and the NVIDIA H100 (Tesla series). These GPUs have 126KB/Streaming Multiprocessor (SM), 192KB/SM, and 256KB/SM of shared memory, respectively. Indeed, one option would be to cache all the wavefronts in shared memory to improve the overall performance since it is the fastest memory available after the registers. However, we decided to keep our design as flexible as possible, supporting all the error rates (i.e., all the possible lengths of the wavefronts). With increasing error rates, the size of the wavefronts grows beyond the physical capacity of the GPU's shared memory; thus, using shared memory to store them becomes infeasible. Therefore, we store the wavefronts required for the alignment and backtracking steps on the GPU main memory using High Bandwidth Memory (HBM) 3 on the H100, HBM 2 on the A100, and GDDR6X on the RTX 3080. The global memory bandwidth of our target GPUs ranges from 19 GB/s (RTX 3080) to 3.35TB/s (H100); thus, we can still maintain high performance. Furthermore, employing global memory for storing the wavefronts allowed us to schedule a bigger number of blocks per kernel call, significantly reducing the resources required by each SM. Our design is entirely device-independent to maintain it as flexible as possible. In particular, we developed the host section of TSUNAMI to detect the total GPU memory availability at runtime; hence, TSUNAMI dynamically schedules the maximum number of alignments to execute in parallel depending on the target GPU it will be executed, fully exploiting inter-task parallelism. The maximum number of alignments scheduled for a kernel call depends on the length of the aligned sequences, their relative error rate, and the available GPU memory. TSUNAMI host application iteratively schedules a different number of alignments per batch (i.e., GPU blocks per kernel call) depending on these factors; the host continuously

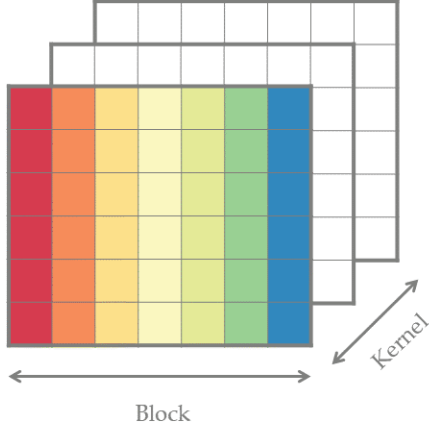


Fig. 3. Representation of Inter-task parallelism. The kernel executes blocks in parallel; each pair of sequences is assigned to a GPU block.

adds alignments to a batch until a batch occupies the entirety of the memory available on the GPU, and then TSUNAMI schedules the kernel execution for that batch and retrieves its corresponding results; this process is iterated, using the same strategy until all the TSUNAMI's scheduled alignments have been performed. Moreover, we also tune the number of scheduled threads per GPU block at every kernel iteration, using the maximum reachable wavefront length as a metric to improve performance and resource utilization. Indeed, the wavefront length depends on the length of the smaller of the two aligned sequences; thus, to optimize the number of parallel elements scheduled, we always spawn a number of threads that is the closest multiple of 32 (*warp size*) to the smaller sequence of a given batch. Indeed, when the number of threads surpasses the wavefront length, several threads will stall in the *Next Wavefront* step. Therefore, scheduling fewer threads will ensure greater resource utilization while enabling the GPU scheduler to compute more blocks in parallel.

C. Backtrace

We implement a similar compact strategy proposed by Haghi *et al.* [22] for the WFA backtrace phase, designing this step to be fully implemented on GPU. The WFA algorithm must store all the wavefronts computed until the last one to correctly perform the backtrace step, outputting the optimal alignment. For this reason, we attach the backtrace operations (i.e., mismatch, insertion, deletion) to each offset computed in the *Next Wavefront* procedure to reduce memory utilization. We store a 32-bit word containing all the operations performed on that particular wavefront offset to reduce memory utilization. Consequently, the word associated with the $M_{s,m-n}$ wavefront contains the compacted optimal alignment. At this point, we have an incomplete CIGAR (i.e., alignment path) containing only mismatch, insertion, and deletion operations (i.e., compact CIGAR). Hence, the kernel transfers the compact CIGAR and the alignment score to the host while the host

TABLE I
EXPERIMENTAL SETUP.

GPU Model	Memory type	Memory capacity	CPU Model	RAM capacity
NVIDIA RTX 3080	GDDR6X	10 GB	Intel Xeon Silver 4208	1 TB
NVIDIA A100	HBM2	40 GB	AMD EPYC 7232P	1 TB
NVIDIA H100	HBM3	80 GB	AMD EPYC 7413	1 TB

computes the full CIGAR completed with match operations. Since this compact backtrace methodology allows the storage of a much smaller number of wavefronts, we were able to better exploit inter-task parallelism, enabling the scheduling of a much higher number of sequences in parallel.

V. DISCUSSION

In this Section, we evaluate the performance of TSUNAMI, comparing it with the State Of the Art software implementation, WFA2-lib [25], and hardware accelerators [22]–[24] of the WFA algorithm. In Section V-A, we present the experimental setup of all the executed tests, while in Section V-B, we show the experimental results obtained using a simulated dataset.

A. Experimental Setup

We collected our results using three different platforms: an NVIDIA GeForce RTX 3080 GPU with 10 GB of GDDR6X connected to an Intel Xeon Silver 4208 processor equipped with 1TB of RAM, an NVIDIA A100 with 40 GB of HBM2 connected with an AMD EPYC 7232P processor and 1TB of RAM, and, finally, an NVIDIA H100 SXM GPU with 80 GB of HBM3 connected with an AMD EPYC 7413 processor with 1TB of RAM (Table I), all experiments were conducted using CUDA 11.8.

We used a synthetic dataset generated with the tool present in the WFA2-lib [25], also employed by the previously developed hardware accelerators [23], [24], [35]. We tested our implementation with different configurations, tuning sequence lengths, and error rates. Hence, we compare our results with the other hardware accelerated implementations in the State Of the Art, namely the ones from Haghi *et al.* [22], Branchini *et al.* [23] and Aguado-Puig *et al.* [24]. To better compare the results of the previously presented implementations, we report the best-attained performance by these designs according to the values indicated in the respective papers. It is important to note that the WFA-GPU [24] implementation is a CPU/GPU hybrid design that offloads to the CPU the alignments that do not fit on the GPU or alignments with an error rate higher than 10%. To better compare TSUNAMI with this solution, we report the execution times of TSUNAMI run using the same GPU employed by Aguado-Puig *et al.*, an NVIDIA GeForce RTX 3080, as well as the NVIDIA A100 and NVIDIA H100. Furthermore, it is important to note that the two FPGA implementations of WFA only report limited information regarding the performance of their designs, showing results with alignment of sequences whose length does not exceed 300 characters.

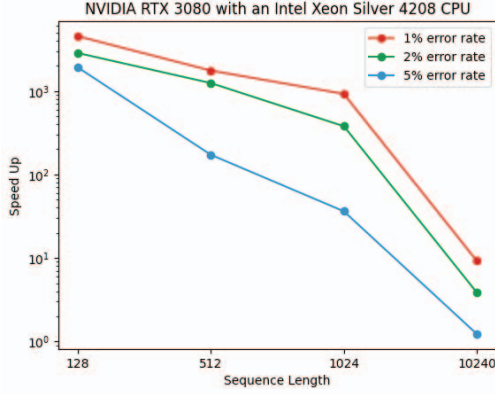


Fig. 4. TSUNAMI speedup over WFA2-lib for different alignment configurations (log-scale). Intel Xeon Silver 4208 CPU with NVIDIA RTX 3080.

TABLE II
TSUNAMI AND WFA2-LIB EXECUTION TIMES IN SECONDS FOR DIFFERENT CONFIGURATIONS OF ALIGNMENTS (INTEL XEON SILVER 4208 CPU WITH NVIDIA RTX 3080).

Sequence length	Number of aligners	Error rate %	TSUNAMI Exec. Time [s]	WFA2-lib Exec. Time [s]
128	100 x 10 ³	1	0.0013	6.0602
512	100 x 10 ³	1	0.0042	7.3031
1024	50 x 10 ³	1	0.0043	4.0059
10240	10 x 10 ³	1	0.3077	2.8445
128	100 x 10 ³	2	0.0024	6.7173
512	100 x 10 ³	2	0.0064	7.9697
1024	50 x 10 ³	2	0.0129	0.0129
10240	10 x 10 ³	2	1.3419	5.1778
128	100 x 10 ³	5	0.0039	7.4525
512	100 x 10 ³	5	0.0657	11.285
1024	50 x 10 ³	5	0.2523	9.140
10240	5 x 10 ³	5	4.4505	5.464

We also compare TSUNAMI with the newly released state-of-the-art software library (i.e., WFA2-lib [25]), indicating both the authors' declared execution times while also reporting the execution times of the library when run on different processors.

B. Experimental Evaluation

To assess the validity of TSUNAMI results, we generated simulated reads with different lengths: short (128 bps), medium (1024 bps), and long (10240 bps). It is important to note that TSUNAMI always reports the correct optimal alignment result (i.e., score and backtrace) that is identical to the one reported by the State Of the Art software, WFA2-lib [25]. We tested TSUNAMI using multiple error rates, in particular 1%, 2%, 5%, and 10%.

Figure 4, Figure 5 and Figure 6 show the speedup achieved by our design evaluated with our three target GPUs using the WFA2-lib [25] library, the represented baseline has been attained running the multi-threaded software on the same server containing the respective GPU. In particular, Table II, Table III, and Table IV illustrate the specific configurations of the

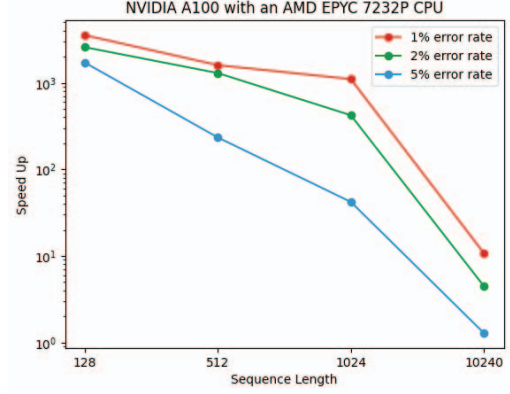


Fig. 5. TSUNAMI speedup over WFA2-lib for different alignment configurations (log-scale). AMD EPYC 7232P CPU with NVIDIA A100.

TABLE III
TSUNAMI AND WFA2-LIB EXECUTION TIMES IN SECONDS FOR DIFFERENT CONFIGURATIONS OF ALIGNMENTS (AMD EPYC 7232P CPU WITH NVIDIA A100).

Sequence length	Number of aligners	Error rate %	TSUNAMI Exec. Time [s]	WFA2-lib Exec. Time [s]
128	500 x 10 ³	1	0.0055	19.384
512	500 x 10 ³	1	0.0116	18.610
1024	50 x 10 ³	1	0.0017	1.8430
10240	10 x 10 ³	1	0.1484	1.6044
128	500 x 10 ³	2	0.0080	20.708
512	500 x 10 ³	2	0.0135	17.469
1024	50 x 10 ³	2	0.0058	2.4607
10240	10 x 10 ³	2	0.6713	3.0213
128	500 x 10 ³	5	0.0108	18.514
512	500 x 10 ³	5	0.1213	28.333
1024	50 x 10 ³	5	0.1141	4.8050
10240	10 x 10 ³	5	4.4369	5.7651

alignments, the TSUNAMI execution times, and the WFA2-lib execution times. For a fair comparison, we used the same sequence length, error rate, and the same number of couples on all three configurations. We can observe that on the server equipped with the NVIDIA RTX 3080; TSUNAMI reaches speedups between 1.22× and 4512.28× against WFA2-lib run using 16 threads on the Intel Xeon Silver 4208. As we can see from Table III and Figure 5, when executing TSUNAMI on an NVIDIA A100 GPU we reached speedups between 1.29× and 3541.63× against WFA2-lib run using 16 threads using an AMD Epyc 7232P processor. Lastly, Figure 6 and Table IV display the performance reached by TSUNAMI with NVIDIA H100 GPU. In this case, TSUNAMI achieves speedups between 3.11× and 3495.66× against software run using 48 threads on an AMD Epyc 7413. As we can see from all of the reported data, we attain the lowest speedup when aligning the longest sequences with the highest error rate. Indeed, the amount of memory required by every aligner increases as the error rate, sequence length, and, consequently, the wavefront length also increases; thus, fewer aligners fit on the GPU at a given time, reducing the number of sequences

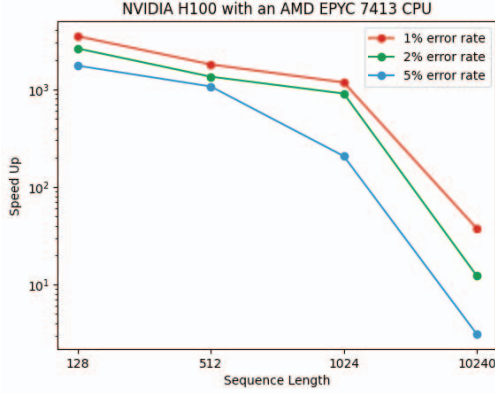


Fig. 6. TSUNAMI speedup over WFA2-lib for different alignment configurations (log-scale). AMD EPYC 7413 24-Core CPU with NVIDIA H100.

TABLE IV
TSUNAMI AND WFA2-LIB EXECUTION TIMES IN SECONDS FOR
DIFFERENT CONFIGURATIONS OF ALIGNMENTS (AMD EPYC 7413
24-CORE CPU WITH NVIDIA H100).

Sequence length	Number of aligners	Error rate %	TSUNAMI Exec. Time [s]	WFA2-lib Exec. Time [s]
128	500 x 10 ³	1	0.0025	8.8879
512	500 x 10 ³	1	0.0067	12.0511
1024	50 x 10 ³	1	0.0012	1.4461
10240	10 x 10 ³	1	0.0518	1.9431
128	500 x 10 ³	2	0.0039	10.409
512	500 x 10 ³	2	0.0116	15.631
1024	50 x 10 ³	2	0.0020	1.8328
10240	10 x 10 ³	2	0.3153	3.8807
128	500 x 10 ³	5	0.0074	12.9473
512	500 x 10 ³	5	0.0161	17.2058
1024	50 x 10 ³	5	0.0118	2.4493
10240	10 x 10 ³	5	2.5310	7.8796

that can be aligned in parallel. Indeed, the higher the amount of memory on the GPU, the higher the speedup we attain. Moreover, the CPU RAM used in every design of TSUNAMI is significantly higher than each GPU memory availability.

In particular, all the processors used in our evaluations are equipped with 1TB of RAM. Therefore, the software results negligibly suffer from the increase in the sequence length. Still, our design always outperforms the software solution on every target GPU. TSUNAMI performance improvements when compared with the software WFA2-lib library range from 1.22× to 4512.28×. Concerning the hardware-accelerated solutions of WFA, we compare our improvements with these implementations in terms of GCUPS, given that this metric is the most used by the community to indicate the performance of a given DNA-alignment algorithm. Conventionally, Cell Updates Per Second (CUPS) indicates the total number of DP-cells updated per second. Even though the WFA algorithm does not require the update of all the cells within the DP-matrix, the community employs the same metric to highlight the performance of WFA against SW and NW based algorithms. Therefore, to better compare TSUNAMI

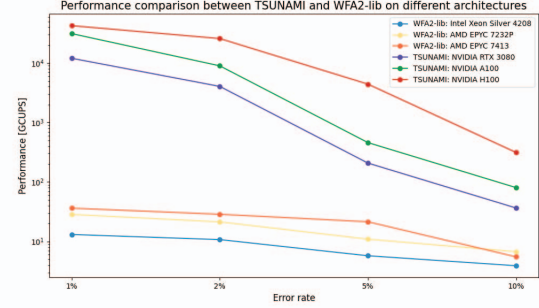


Fig. 7. Performance (log-scale) comparison between TSUNAMI and WFA2-lib for alignments of 1024 bases with different error rates on multiple architectures.

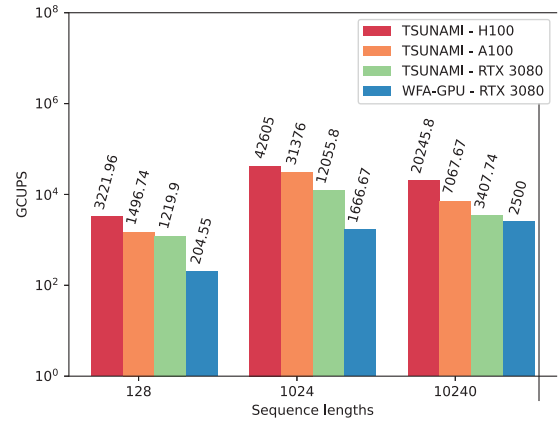


Fig. 8. Comparison between TSUNAMI and WFA-GPU for various alignment configurations (log-scale). Numbers over the bars are the specific values of the GCUPS for each type of genomic sequence and architecture.

with all the previously presented implementations, we adopted the same computation of CUPS as the other works present in literature [23], [24], [32], [35], applying the following equation:

$$GCUPS = \frac{\text{len}(q) \times \text{len}(t) \times \text{num_couples}}{\text{time} \times 10^9} \quad (3)$$

First, we compared TSUNAMI with WFA2-lib [11] implementation; we represented their performances in terms of GCUPS in Figure 7. From this plot, we can appreciate the difference in performance between our implementation on multiple GPU devices and the multi-threaded software. As we can see, TSUNAMI's performance outperforms WFA2-lib [11] by three orders of magnitude with a 1% error rate and by over one order of magnitude with a 10% error rate. We also compared TSUNAMI with WFA, WFA-GPU [24]. Figure 8 compares our implementation with WFA-GPU using our target GPUs: NVIDIA H100, NVIDIA A100, and the NVIDIA RTX 3080. The design configurations reported in Figure 8, from left to right, are the following: short (~128),

TABLE V
PEAK PERFORMANCE EVALUATION

Paper	Architecture	Platform	Peak performance [GCUPS]	Speedup wrt WFA2-lib [11]	Speedup wrt FPGA [23]
WFA2-lib [11]*	CPU	Intel Xeon Platinum	116.27	1×	0.04×
[22]	FPGA	Xilinx Virtex Ultrascale+	1251.7	10.77×	0.44×
[22]	FPGA	2 x Xilinx Virtex Ultrascale+	2073.7	17.84×	0.72×
[23]**	FPGA	Xilinx Alveo U250	2876	24.74×	1×
WFA-GPU [24]*	CPU+GPU	Intel Xeon + NVIDIA RTX 3080	2500	21.5×	0.87×
TSUNAMI	GPU	NVIDIA RTX 3080	12055.79	103.69×	4.19×
TSUNAMI	GPU	NVIDIA A100	31375.97	269.85×	10.91×
TSUNAMI	GPU	NVIDIA H100	42604.98	366.43×	14.81×

TABLE VI
PERFORMANCE EVALUATION OF TSUNAMI AND WFA2-LIB ALIGNING 500K READS WITH ERROR RATES > 10% USING AN NVIDIA H100 (80GB) GPU FOR TSUNAMI AND TWO AMD EPYC 7J13 (256 THREADS IN TOTAL) FOR WFA2-LIB

Sequence Length	Error Rate	TSUNAMI Execution Time [ms]	TSUNAMI [GCUPS]	WFA2-lib [11] Execution Time [ms]	WFA2-lib [11] [GCUPS]	Speedup
128	12	0.01	646.23	32.71	0.25	2580.32×
512	12	0.05	2398.72	41.81	3.14	765.07×
1024	12	5.69	92.09	55.51	9.45	9.75×
128	14	0.01	592.8	33.89	0.24	2452.23×
512	14	0.11	1166.62	43.67	3	388.67×
1024	14	9.65	54.32	60.07	8.73	6.33×
128	15	0.02	437.37	33	0.25	1761.73×
512	15	0.41	321.22	42.67	3.07	104.57×
1024	15	11.41	45.93	61.14	8.57	5.36×
128	18	0.03	313.87	34.82	0.24	1333.98×
512	18	0.2	652.06	46.54	2.82	231.55×
1024	18	17.25	30.04	71.1	7.37	4.12×

medium (~ 1024), long (~ 10240) reads, and have an error rate of 2%. We used the NVIDIA RTX 3080 since it is the same GPU used by the WFA-GPU implementation, allowing us to show the improvements attained by our design, regardless of the employed GPU. We also reported results executing on A100 and H100 NVIDIA GPUs to show how our work is also able to scale with modern architectures. Our design significantly outperforms the WFA-GPU implementation on all the GPU architectures, highlighting how our design is much more efficient when compared to WFA-GPU. Our implementation always outperforms WFA-GPU; furthermore, we noticed that TSUNAMI follows the same behavior we previously observed when comparing it with WFA2-lib. Indeed, it is important to note that the WFA-GPU design offloads to the CPU the computations of alignments of sequences that do not fit in the GPU; thus, the behavior of WFA-GPU mirrors the one of WFA2-lib for very long and error-prone alignments rendering TSUNAMI both faster and more reliable in this instances. We also compare TSUNAMI with the two FPGA designs in the State Of the Art. To uniform the comparison within these accelerators, we report the peak performance of all the hardware accelerators of the WFA algorithm in Table V. The reported GCUPS are the optimal ones reported by each work in the literature, including ours. In particular, *Branchini et al.* [23] attained their best-performing configuration by aligning 10M couples of 100 bases in each sequence with an error rate of 1%. *Haghi et al.* [22] only report the peak performance in GCUPS

without reporting the alignment configuration used to attain that result. *Aguado-Puig et al.* [24] GCUPS best configuration employs 100K alignments of 10K base pairs with a 2% error rate. TSUNAMI's peak performance comes from the alignment of 50K sequences whose length is 1024 bases and with an error rate of 1%. In particular, we can observe that TSUNAMI's overall performance improvements range from 14.81×, when compared with the best-performing FPGA design, up to 366.43× when compared with WFA2-lib. Moreover, TSUNAMI is capable of aligning sequences with error rates bigger than 10% without offloading its computation to the CPU as WFA-GPU [24] while still significantly outperforming the literature software solution. We report TSUNAMI's performance for these alignments in Table VI, comparing its performance against WFA2-lib [11] aligning 500K sequences.

VI. TSUNAMI ROOFLINE ANALYSIS

In this Section, we provide a detailed analysis of the optimized TSUNAMI GPU performance by exploiting the Roofline model [15], [36]–[38] to model our specific implementation.

The visually intuitive Berkeley Roofline model utilizes a *bound and bottleneck* analysis approach to aid in comprehending the performance of a specific kernel. It outlines the various factors that affect computer system performance and correlates processor performance with off-chip memory traffic. By characterizing a kernel's performance in Warp Billions

Instructions Per Second (GIPS) as a function of its Operational Intensity (OI) on a 2D log-log scale graph, the Roofline model incorporates integer performance, operational intensity, and memory bandwidth. To achieve this, the model employs the X -axis to represent OI, which measures instructions per byte of Global Memory (GMEM) traffic, and the Y -axis to represent Warp GIPS, given that the kernel only performs integer operations. Figure 9 shows the performance of our kernel depicted using the Roofline model. On our target GPU, the NVIDIA Tesla H100, we have 114 SMs available, where each SM consists of four processing blocks, called *warp schedulers*. Each warp scheduler within each block of the SM can dispatch only one instruction per cycle. As such, the theoretical maximum (warp-based) of instruction/s is $114 \text{ SM} \times (4 \text{ Warp schedulers per SM}) \times (1 \text{ instruction/cycle}) \times 1.095 \text{ GHz} = 499.32 \text{ GIPS}$. Furthermore, each processing SM contains 32 FP32 cores, 16 FP64 cores, and 16 INT32 cores. The peak integer performance for the NVIDIA H100 is then $16/32 \times 499.32 = 249.66 \text{ integer Warp GIPS}$ since 16 INT32 cores can only support 16 threads out of 32 threads in one warp. The maximum performance is upper bounded by both the theoretical INT32 peak rate and the peak memory bandwidth, defined by the red line in Figure 9. This maximum attainable performance represents a ceiling in the Roofline model plot for the considered GPU platform and is independent of the executed algorithm. A kernel is compute-bound if limited by the hardware performance limit (blue line/light blue area), and it is memory-bound if the Warp GIPS are limited by the memory bandwidth (red line/red area). To adapt the Roofline ceiling to the WFA algorithm, we exploited the same methodology described by *Zeni et al.* [15] for a similar alignment algorithm. Indeed, given that the number of parallel elements that we can compute in parallel depends on the iteration of WFA, we can describe the new peak with:

$$\text{Ceiling} = \frac{1}{N} \sum_{i=1}^N \frac{f \times N_{op,i} \times B}{\lceil (T \times B) / MAX_R \rceil} \quad (4)$$

Equation (4) defines the new maximum performance by averaging the number of cells that GPU can compute in parallel. N indicates the total number of parallel iterations for a given algorithm, f is the theoretical ceiling (249.66 warp GIPS), B is the number of scheduled blocks, $N_{op,i}$ indicates the number of operations that need to be computed at each iteration, T is the number of scheduled threads per block, and, finally, MAX_R indicates the number of INT32 cores available. We can observe TSUNAMI's overall performance behavior is shown in Figure 9. Results show the operational intensity of our kernel on the HBM memory of the GPU, indicating that our implementation is not memory bound and that we are bound by the theoretical peak performance ceiling. In other words, the operational intensity of our design is high enough to be in the compute-bound area of the Roofline model; thus, it is not limited by the GMEM memory bandwidth. Note that the optimized performance of our algorithm is very close to the adapted theoretical ceiling. Considering that the adapted

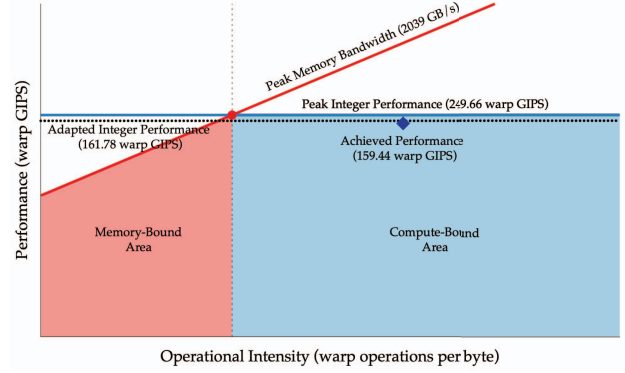


Fig. 9. Roofline analysis for our kernel on the NVIDIA Tesla H100 GPU performing 500K alignments of 1024 character reads with error rate of 1%.

ceiling does not take into account memory latency, the results of our implementation are extremely close to the maximum achievable performance. Therefore, TSUNAMI represents a near-optimal implementation of the WFA algorithm, and it is only limited by the compute capability of the GPU.

VII. CONCLUSION AND FUTURE WORK

This work presents TSUNAMI, a high-performance GPU-optimized design of the WaveFront Alignment algorithm. WFA is employed in several important genomics applications; however, it is particularly challenging for GPU parallelization due to its highly dynamic nature. Detailed results and analyses show significant performance acceleration using our design. TSUNAMI achieves speedups up to $4512.28\times$ when compared with the State Of the Art software implementation. Additionally, TSUNAMI also outperforms all the other hardware accelerated solutions of WFA present in the literature. In particular, TSUNAMI performance improvements reach up to $14.81\times$ when compared with the FPGA state-of-the-art design and up to $17.04\times$ when compared with the best GPU implementation. Finally, we performed a roofline analysis of our implementations, demonstrating that our WFA design methodology results in near-optimal performance. Our overall results show that our optimized kernel is very flexible and efficient and that TSUNAMI is a very capable candidate to be integrated into multiple genomic pipelines. Future works will explore novel WFA memory modes (i.e., BiWFA [32]) to reduce memory utilization. Furthermore, we plan on expanding TSUNAMI to run on multiple GPUs while also looking at its integration within multiple pipelines that currently exploit SW or NW derived algorithms.

ACKNOWLEDGMENTS

The Authors would like to thank the NVIDIA University Program for the hardware donations and Oracle Research Program for the Oracle Cloud Credits.

A. Artifact Appendix

A.1 Abstract

TSUNAMI is a genome alignment application that implements the Wavefront alignment algorithm on GPU. We collected our results using three different GPUs: NVIDIA RTX 3080 connected to an Intel Xeon Silver 4208 processor equipped with 1TB of RAM, NVIDIA A100 connected with an AMD EPYC 7232P processor and 1TB of RAM, and NVIDIA H100 connected with an AMD EPYC 7413 processor with 1TB of RAM. Nonetheless, any CUDA-enabled NVIDIA GPU can be used to check TSUNAMI result correctness. To assess the correctness of our implementation, we compare our alignment results with the ones of the original software implementation. When executed TSUNAMI displays a self-explaining message communicating the correctness of the alignments. TSUNAMI outputs GCUPS and execution times of the GPU and CPU implementations. We expect that the above-mentioned outputs match the results reported in the paper. TSUNAMI's code is publicly available on a Github repository, together with instructions on how to compile it and run it. We invite the reviewers to reach out to us for any concern.

A.2 Artifact check-list (meta-information)

- **Algorithm:** WaveFront Alignment algorithm
- **Program:** wfa_gpu
- **Compilation:** Through provided Makefile, C++14 and CUDA 11.8 (or greater) required.
- **Data set:** Tool generated.
- **Run-time environment:** CUDA required, Linux (Ubuntu 20.04 is recommended).
- **Hardware:** NVIDIA RTX 3080, NVIDIA A100, NVIDIA H100 (to check performance numbers on the paper), any NVIDIA GPU to check correctness.
- **Run-time state:** NVIDIA runtime environment has to be loaded
- **Execution:** A script is present on the provided repository for all the experiments, but additional experiments can be performed following the repository README.
- **Metrics:** Sequence inputs and scoring option are the input metrics.
- **Output:** GCUPS of GPU and CPU implementations, GPU and CPU execution times, and results correctness.
- **Experiments:** Executing software and GPU accelerated software to compare results correctness and performance.
- **How much disk space required (approximately):** 8GB (Ubuntu + CUDA 11.8 + repo/inputs space)
- **How much time is needed to prepare workflow (approximately):** 1 hour if CUDA is not installed, otherwise 5 minutes
- **How much time is needed to complete experiments (approximately):** 15 minutes
- **Publicly available:** Yes, available here https://github.com/giuliagerometta/TSUNAMI_PACT.git
- **Code licenses (if publicly available):** MIT License.
- **Data licenses (if publicly available):** None, the input dataset is generated using users specific inputs.
- **Workflow framework used:** None.
- **Archived (provide DOI):** <https://doi.org/10.5281/zenodo.8348893>

A.3 Description

A.3.1 How to access

The code can be accessed on the Github repo: https://github.com/giuliagerometta/TSUNAMI_PACT.git. If needed, the au-

thors can provide access to a server with an NVIDIA A100. To access it please communicate with the authors.

A.3.2 Hardware dependencies

NVIDIA RTX 3080, NVIDIA A100, NVIDIA H100 (to check performance numbers on the paper), any NVIDIA GPU to check correctness. CPU performance depends on the server's mounted CPU.

A.3.3 Software dependencies

CUDA 11.8 (or greater), C++14

A.3.4 Data sets

Tool generated

A.4 Installation

Follow this guide to install CUDA <https://developer.nvidia.com/cuda-downloads>. The code can simply be installed using the README instructions through a provided Makefile.

A.5 Experiment workflow

Manually run the experiments provided in our work following the readme or run a provided bash file that already contains the experiments to be run. Each experiment will provide correctness and performance info, if run through the bash script, information regarding the inputs of the experiments will also be provided.

A.6 Evaluation and expected results

The performance results should match the ones provided in the paper together with matching the correctness of the software implementation.

A.7 Experiment customization

Custom and additional experiments can be performed following the README guidance for generating custom inputs.

A.8 Notes

If the committee is unable to attain machines for the experiments please reach out to the authors, we can provide continuous access to a server with an A100 and pre-installed environment, but we cannot provide direct access to H100 and RTX 3080 equipped machines, as we used on-demand services for those. Ready to use scripts optimized for multiple machines are available in the GitHub repository, these scripts can be used to reproduced the results shown in the paper, while users can use the available with personalized options for custom workloads. If there is any difficulty in executing the code please feel free to reach out to us.

A.9 Methodology

Submission, reviewing and badging methodology:

- <https://www.acm.org/publications/policies/artifact-review-and-badging-current>
- <http://cTuning.org/ae/submission-20201122.html>
- <http://cTuning.org/ae/reviewing-20201122.html>

REFERENCES

- [1] S. B. Needleman and C. D. Wunsch, "A general method applicable to the search for similarities in the amino acid sequence of two proteins," *Journal of molecular biology*, vol. 48, no. 3, pp. 443–453, 1970.
- [2] J. T. Simpson, K. Wong, S. D. Jackman, J. E. Schein, S. J. Jones, and I. Birol, "Abyss: a parallel assembler for short read sequence data," *Genome research*, vol. 19, no. 6, pp. 1117–1123, 2009.
- [3] S. Koren, B. P. Walenz, K. Berlin, J. R. Miller, N. H. Bergman, and A. M. Phillippy, "Canu: scalable and accurate long-read assembly via adaptive k-mer weighting and repeat separation," *Genome research*, vol. 27, no. 5, pp. 722–736, 2017.
- [4] B. Langmead and S. L. Salzberg, "Fast gapped-read alignment with bowtie 2," *Nature methods*, vol. 9, no. 4, pp. 357–359, 2012.
- [5] H. Li, "Aligning sequence reads, clone sequences and assembly contigs with bwa-mem," *arXiv preprint arXiv:1303.3997*, 2013.
- [6] —, "Minimap2: pairwise alignment for nucleotide sequences," *Bioinformatics*, vol. 34, no. 18, pp. 3094–3100, 2018.
- [7] M. A. DePristo, E. Banks, R. Poplin, K. V. Garimella, J. R. Maguire, C. Hartl, A. A. Philippakis, G. Del Angel, M. A. Rivas, M. Hanna *et al.*, "A framework for variation discovery and genotyping using next-generation dna sequencing data," *Nature genetics*, vol. 43, no. 5, pp. 491–498, 2011.
- [8] A. McKenna, M. Hanna, E. Banks, A. Sivachenko, K. Cibulskis, A. Kernysky, K. Garimella, D. Altshuler, S. Gabriel, M. Daly *et al.*, "The genome analysis toolkit: a mapreduce framework for analyzing next-generation dna sequencing data," *Genome research*, vol. 20, no. 9, pp. 1297–1303, 2010.
- [9] T. F. Smith, M. S. Waterman *et al.*, "Identification of common molecular subsequences," *Journal of molecular biology*, vol. 147, no. 1, pp. 195–197, 1981.
- [10] O. Gotoh, "An improved algorithm for matching biological sequences," *Journal of molecular biology*, vol. 162, no. 3, pp. 705–708, 1982.
- [11] S. Marco-Sola, J. C. Moure, M. Moreto, and A. Espinosa, "Fast gap-affine pairwise alignment using the wavefront algorithm," *Bioinformatics*, vol. 37, no. 4, pp. 456–463, 2021.
- [12] E. W. Myers, "An o (nd) difference algorithm and its variations," *Algorithmica*, vol. 1, no. 1-4, pp. 251–266, 1986.
- [13] G. Myers, "Efficient local alignment discovery amongst noisy long reads," in *Algorithms in Bioinformatics: 14th International Workshop, WABI 2014, Wrocław, Poland, September 8-10, 2014. Proceedings 14*. Springer, 2014, pp. 52–67.
- [14] A. Chacón, S. Marco-Sola, A. Espinosa, P. Ribeca, and J. C. Moure, "Thread-cooperative, bit-parallel computation of levenshtein distance on gpu," in *Proceedings of the 28th ACM international conference on Supercomputing*, 2014, pp. 103–112.
- [15] A. Zeni, G. Guidi, M. Ellis, N. Ding, M. D. Santambrogio, S. Hofmeyr, A. Buluç, L. Olikar, and K. Yelick, "Logan: High-performance gpu-based x-drop long-read alignment," in *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2020, pp. 462–471.
- [16] Y. Liu, T.-T. Tran, F. Lauenroth, and B. Schmidt, "Swaphi-ls: Smith-waterman algorithm on xeon phi coprocessors for long dna sequences," in *2014 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 2014, pp. 257–265.
- [17] A. Zeni, G. W. Di Donato, L. Di Tucci, M. Rabozzi, and M. D. Santambrogio, "The importance of being x-drop: High performance genome alignment on reconfigurable hardware," in *2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2021, pp. 133–141.
- [18] A. Zeni, M. Crespi, L. Di Tucci, and M. D. Santambrogio, "An fpga-based computing infrastructure tailored to efficiently scaffold genome sequences," in *2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2019, pp. 333–333.
- [19] A. Zeni, F. Peverelli, E. Cabri, L. Di Tucci, L. Cerina, and M. D. Santambrogio, "circa: a fpga-based circular rna aligner," in *2019 IEEE EMBS International Conference on Biomedical & Health Informatics (BHI)*. IEEE, 2019, pp. 1–4.
- [20] A. Zeni, G. W. Di Donato, A. Della Valle, F. Carloni, and M. D. Santambrogio, "On the genome sequence alignment fpga acceleration via ksw2z," in *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2023, pp. 1–5.
- [21] A. Zeni, E. Del Sozzo, B. Branchini, L. Di Tucci, and M. D. Santambrogio, "New solution for a (scaff) old problem: an fpga approach," in *2023 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 2023, pp. 75–78.
- [22] A. Haghi, S. Marco-Sola, L. Alvarez, D. Diamantopoulos, C. Hagleitner, and M. Moreto, "An fpga accelerator of the wavefront algorithm for genomics pairwise alignment," in *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 2021, pp. 151–159.
- [23] B. Branchini, G. Gerometta, L. Cicolini, A. Zeni, E. Del Sozzo, and M. D. Santambrogio, "Surfing the wavefront of genome alignment," in *2022 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2022, pp. 1754–1758.
- [24] Q. Aguado-Puig, S. Marco-Sola, J. C. Moure, C. Matzoros, D. Castells-Rufas, A. Espinosa, and M. Moreto, "Wfa-gpu: Gap-affine pairwise alignment using gpus," *bioRxiv*, pp. 2022–04, 2022.
- [25] S. Marco-Sola, "WFA2-lib," 2021. [Online]. Available: <https://github.com/smarco/WFA2-lib>
- [26] W. R. Pearson, "Searching protein sequence libraries: comparison of the sensitivity and selectivity of the smith-waterman and fasta algorithms," *Genomics*, vol. 11, no. 3, pp. 635–650, 1991.
- [27] L. Pipes and R. Nielsen, "Ancestralclust: clustering of divergent nucleotide sequences by ancestral sequence reconstruction using phylogenetic trees," *Bioinformatics*, vol. 38, no. 3, pp. 663–670, 2022.
- [28] Y. Yan, N. Chaturvedi, and R. Appuswamy, "Accel-align: a fast sequence mapper and aligner based on the seed-embed-extend method," *BMC bioinformatics*, vol. 22, no. 1, pp. 1–20, 2021.
- [29] B. Song, S. Marco-Sola, M. Moreto, L. Johnson, E. S. Buckler, and M. C. Stitzer, "Anchorwave: sensitive alignment of genomes with high diversity, structural polymorphism and whole-genome duplication variation," *bioRxiv*, pp. 2021–07, 2021.
- [30] J. M. Eizenga and B. Paten, "Improving the time and space complexity of the wfa algorithm and generalizing its scoring," *bioRxiv*, pp. 2022–01, 2022.
- [31] M. Patterson, T. Marschall, N. Pisanti, L. Van Iersel, L. Stougie, G. W. Klau, and A. Schönhuth, "Whatshap: weighted haplotype assembly for future-generation sequencing reads," *Journal of Computational Biology*, vol. 22, no. 6, pp. 498–509, 2015.
- [32] S. Marco-Sola, J. M. Eizenga, A. Guaracino, B. Paten, E. Garrison, and M. Moreto, "Optimal gap-affine alignment in o (s) space," *Bioinformatics*, vol. 39, no. 2, p. btad074, 2023.
- [33] S. Diab, A. Nassereldine, M. Alser, J. Gómez Luna, O. Mutlu, and I. El Hajj, "A framework for high-throughput sequence alignment using real processing-in-memory systems," *Bioinformatics*, vol. 39, no. 5, p. btad155, 2023.
- [34] D. Castells-Rufas, "Gpu acceleration of levenshtein distance computation between long strings," *Parallel Computing*, vol. 116, p. 103019, 2023.
- [35] Q. Aguado-Puig, S. Marco-Sola, J. C. Moure, D. Castells-Rufas, L. Alvarez, A. Espinosa, and M. Moreto, "Accelerating edit-distance sequence alignment on gpu using the wavefront algorithm," *IEEE access*, vol. 10, pp. 63 782–63 796, 2022.
- [36] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Commun. ACM*, vol. 52, no. 4, 2009.
- [37] N. Ding and S. Williams, "An instruction roofline model for gpus," *2019 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, 2019.
- [38] S. Williams, A. Waterman, and D. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, 2009.